JFEM TACS Back-End Report

Current Status, Usage, Validation, and Engineering Potential

JFEM

Universidad de Sevilla

Updated June 12, 2026

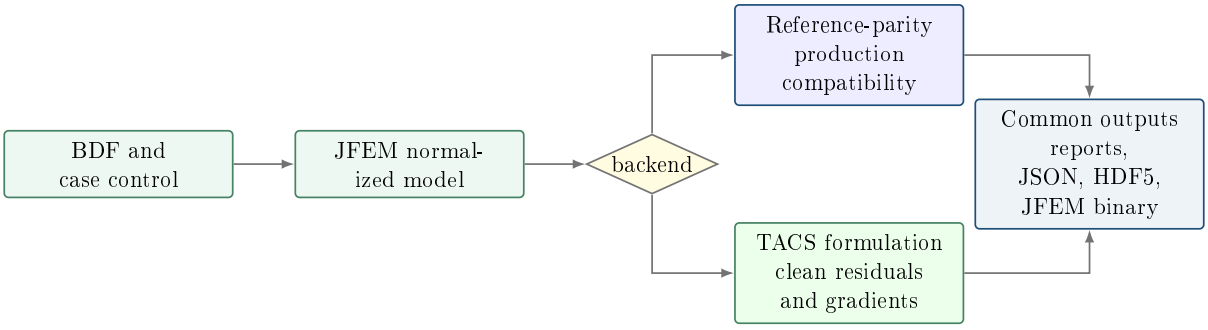
Executive Summary

JFEM now has a dual-backend architecture. The default `reference\_parity` backend preserves the calibrated compatibility path, while the `tacs\_formulation` backend provides a residual-first shell formulation inspired by the architecture of TACS: clean element residuals, explicit tangents, backend-local derivative hooks, and SOL200-lite routes that can consume those gradients.

The TACS back end is not a runtime dependency on TACS. It is a Julia implementation inside JFEM that imports useful formulation ideas: residual and tangent first, constitutive abstraction, adjoint-friendly sensitivities, and a clear boundary between the BDF/SOL200 front end and the physics engine.

As of this update, the guarded TACS shell scope includes `CQUAD4`, `CQUADR`, and `CTRIA3`; `PSHELL/MAT1` and `PCOMP/MAT8` CLT constitutives; SOL101, SOL103, SOL105, SOL106, and SOL200-lite routes; plus thickness, ply, material, displacement, buckling, mass, and KS von-Mises stress sensitivities on supported decks.

1 Strategic Role



The design intent is a common input and output workflow:

- Use the same BDF reader, INCLUDE resolver, case-control parser, and model builder.
- Select the back end through model metadata, manifest metadata, or `JFEM\_BACKEND`.
- Preserve the default as `reference\_parity` unless a specific card or route requires the TACS formulation.
- Keep SOL200-lite problem translation backend-neutral where possible; let the selected backend provide physics and derivatives.

2 How to Use It

Default behavior

No setting is required for the parity path:

```
julia -project=.  run\_jfem.jl model.bdf
```

The default backend is `reference\_parity`. Result payloads include backend metadata when the solve path attaches it.

Opt into the TACS formulation

Set the environment variable:

```
$env:JFEM\_BACKEND='tacs\_formulation'; julia -project=.  run\_jfem.jl model.bdf
```

or set the model field:

```
model["backend"] = "tacs\_formulation"
```

Batch manifests and the Python client also carry backend metadata, so the same deck can be run through either backend in automated campaigns.

Automatic CQUADR routing

`CQUADR` is treated specially by design. If a model contains a `CQUADR` shell element, `solve\_model(model)` forces the actual solver route to `tacs\_formulation`, even when the requested backend is the default `reference\_parity`. The result records:

- `requested\_backend = "reference\_parity"`
- `backend = "tacs\_formulation"`
- `backend\_forced\_by = "CQUADR"`

This lets upstream TACS-style decks retain their native `CQUADR` spelling while still running through a formulation compatible with the TACS slice.

3 Implemented Capability

Area	Current status
Backend boundary	`ReferenceParityBackend` and `TACSFormulationBackend` are explicit public backend types.
Selection	Environment, model metadata, manifest plumbing, and Python-client plumbing can select the backend.
Supported shells	TACS slice supports `CQUAD4`, `CQUADR`, and `CTRIA3` in the guarded shell path.
Constitutives	`PSHELL/MAT1` and `PCOMP/MAT8` CLT. PCOMP refresh is tied to active material data.
SOL101	Residual-first shell stiffness/residual, static compliance and displacement adjoints, thickness and design-variable gradients.
SOL103	TACS shell stiffness with shared mass assembly and shared modal packaging.
SOL105	Static preload, native residual-first geometric stiffness, eigenvalue route, and buckling thickness sensitivity hook.
SOL106	Backend nonlinear-state callback for tangent-operator and guarded formal flat-shell von-Karman states.
SOL200-lite	Shared route surface with TACS sensitivities for thickness, PCOMP plies, material variables, buckling, displacement, mass, and stress responses.

4 SOL200-lite Design Routes

The TACS back end is most useful when gradients matter. Current SOL200-lite coverage includes:

- Shell-thickness sizing for compliance, displacement constraints, and buckling objectives.
- PCOMP ply-level thickness and angle variables.
- Material design variables for `MAT1 E/G/NU/RHO` and `MAT8 RHO`.
- MAT8 laminate stiffness variables: `DVMREL1,MAT8,E1`, `E2`, `G12`, and `NU12`.
- KS von-Mises stress response objectives and stress-constrained mass minimization routes on supported shell decks.

For MAT8 stiffness changes, affected PCOMP CLT matrices are recomputed from the active material table. This is important for laminate optimization: changing a material field is not just a scalar update; it changes transformed ply stiffness, shear terms, and laminate stiffness matrices.

5 Validation Evidence

Guard	What it checks
Default backend	backend normalization and explicit selection behavior.
Parity smoke	bundled SOL101/SOL103/SOL105 examples still solve through parity by default.
Manifest metadata	manifest and exported JSON preserve backend metadata.
SOL101 derivatives	residual, tangent, thickness derivative, compliance/displacement adjoints, and PCOMP ply derivatives.
SOL103	CQUAD4 and CTRIA3 modal routes with finite modes and symmetry checks.
SOL105	buckling route, nonzero native geometric stiffness, and derivative agreement.
SOL106	tangent and formal nonlinear callback paths on flat-shell decks.
SOL200-lite	route suite including material, PCOMP, buckling, displacement, and stress routes.
CQUADR	generated CQUADR SOL101 deck requests default parity and is solved by TACS.
CQUADR expanded routes	generated CQUADR SOL103, SOL105, and SOL200-lite decks request default parity and run through TACS.
CQUADR SOL106	generated CQUADR nonlinear decks request default parity and run through TACS tangent and formal callbacks.
CQUADR mixed shell	generated CQUAD4/CQUADR/CTRIA3 patch requests default parity and runs through TACS static, modal, and buckling routes.
CQUADR multi-property	generated two-property CQUAD4/CQUADR/CTRIA3 patch checks PID-separated compliance, displacement, buckling, and SOL200 grouped sizing routes.
Static audit	closure guard for backend hooks, route guards, metadata, and public plumbing.

Selected numerical results

The current guarded evidence includes:

- SOL101 global tangent relative error: `2.70e-10`.
- SOL101 global `dK/dt` relative error: `1.32e-10`.
- Compliance `dC/dt` relative error: `5.60e-8`.
- Displacement `dU/dt` relative error: `5.31e-8`.
- SOL105 Rayleigh/finite-difference derivative relative error: `5.02e-13`.
- SOL105 public backend hook relative error: `3.27e-9`.
- airframe SOL105 parity guard retained under the `3.0 percent` threshold.
- MAT8 E1/E2/G12/NU12 material route checks report `tacs\_formulation\_design\_tangent` gradients and finite updated design variables.
- CQUADR forced-route guard passed. The generated deck records:
  - requested backend: `reference\_parity`;
  - actual backend: `tacs\_formulation`;
  - forced-route marker: `backend\_forced\_by = CQUADR`.

- CQUADR expanded-route guard passed for SOL103, SOL105, and SOL200-lite. Representative generated results include first modal frequency `12.85812965362194 Hz`, first buckling factor `88.11485003708977`, buckling `dLambda/dt 13216.31733100194`, and SOL200-lite objective `0.22697649705562117`.
- CQUADR SOL106 guard passed for both nonlinear callbacks. The generated standard and formal decks record actual backend `tacs\_formulation`; representative diagnostics include `Kg nnz = 576`, `K` symmetry error `0.0`, callback `tacs\_formulation\_tangent\_operator`, and formal callback `tacs\_formulation\_formal\_shell\_von\_karman`.
- CQUADR mixed-shell guard passed on a generated two-by-two patch with two CQUAD4 elements, one CQUADR element, and two CTRIA3 elements. Representative values: SOL101 gradient `-298.741593`, SOL103 `f1 = 3.831795 Hz`, SOL105 `lambda1 = 32.661785`, and `Kg nnz = 1692`.
- CQUADR multi-property guard passed on a generated two-property patch. Representative PID-separated values: compliance gradients `-357.187281` and `-14.787197`; displacement gradients `1.533854` and `0.050263`; SOL105 `lambda1 = 29.897826`; buckling gradients `5109.286330` and `361.728919`; SOL200 objective `2.018400`.

6 What It Is Useful For

Engineering use

- Compare production-style parity solves with clean formulation solves from the same BDF/SOL200 source.
- Study shell sizing, laminate ply variables, material stiffness variables, buckling sensitivities, and stress-constrained mass problems.
- Keep reports and exported payloads backend-aware, which is essential for validation campaigns and audit trails.
- Run TACS-origin decks that use `CQUADR` without mechanically rewriting them to `CQUAD4`.

Development use

- Add new shell formulation features behind one backend boundary instead of mixing them into the parity switchboard.
- Use finite-difference and AD checks as guard rails for derivative development.
- Grow SOL200-lite coverage through backend-neutral syntax while keeping physics and sensitivities backend-specific.
- Build a controlled research platform for adjoints, laminate optimization, and stress/buckling constraints.

7 Current Limits

The current TACS back end is deliberately guarded. It is not yet:

- a runtime dependency on external TACS;

- a full replacement for every Reference-parity element family;
- a complete industrial optimizer;
- validated over a large external TACS/Reference production campaign;
- intended to remove the `reference\_parity` path, which remains the default compatibility route.

The best mental model is: parity remains the drop-in compatibility engine; TACS formulation is the clean derivative and optimization engine for the guarded shell subset.

8 Near-Term Recommendations

1. Keep `reference\_parity` as the default until larger production validation has been completed.
2. Extend the new CQUADR mixed-shell and multi-property guards to CRM-style modal decks and larger production-like mixed shell models.
3. Add more MAT8 stiffness route cases for mixed material relations and multiple PCOMP properties.
4. Run the full SOL200 route guard periodically with a large timeout, because the full suite is intentionally broader than the material-only and stress-only filters.
5. Preserve the common BDF/SOL200 front end. New TACS-formulation physics should enter behind backend hooks, not by forking the input language.

9 Bottom Line

The TACS back end gives JFEM a clean design-analysis path while keeping the Reference-compatible workflow. It makes the solver more useful for laminate optimization, shell sizing, stress constraints, buckling studies, and future adjoint-based campaigns. Its value is not that it copies TACS; its value is that it brings the strongest TACS architectural ideas into JFEM's own Julia-native solver stack.